



Chains with LangGraph

What is a Chain?

A chain is a sequence of nodes connected by edges. Each node can be a simple function or a more complex node that interacts with the environment. The chain is executed sequentially, and the output of one node is passed to the next. The chain can be executed synchronously or asynchronously.

How the Graph Group, directs the workflow

The first graph is simply START goes to the LLM node, which goes to END. It groups the LLM node into a graph and then handles the conversation state. There is no tool execution here; if the model produces a tool call, it just sits in the final output unexecuted.

The node holds the graph in two stages, and it helps to see this as discrete progression.

The second graph adds the tool node and the conditional edge that when the model produces a tool call, the graph routes to the tool node and then the result, when the model gives a direct response, the graph ends immediately. The same LLM node handles both cases; only the routing changes.

Four Concepts This Code Introduces

The notebook builds one idea on top of another, understanding why each piece exists makes the code obvious.

1. Chat Messages as Nodes

The graph needs to remember the conversation. LangGraph uses LangChain's `BaseChatModel` to handle the conversation state. The chat model is a stateful object that can be updated with new messages and can return the next message. The chat model is used to generate the next message in the conversation.

2. Chat Nodes as Graph Nodes

The LLM is not called inside the graph. It lives inside a node. The node is a function that takes the current state and returns the next state. The node is called by the graph engine. The node is responsible for updating the chat model with the latest messages and for generating the next message. The node is also responsible for handling the tool calls and for routing the graph to the appropriate tool node.

3. Handling Tools in the LLM

An LLM on its own only generates text. Tools let it interact with the outside world, such as calling an API, querying a database, or fetching a calendar entry. The tool node is a function that takes the tool call from the LLM and returns the result. The tool node is called by the graph engine. The tool node is responsible for executing the tool call and for returning the result to the LLM node.

4. Reasoning, Tool Calls, and Final Answer

The model reasoning is a tool and the tool actually reasoning are two separate steps. LangGraph provides `Reason` to handle reasoning. The `Reason` node is a function that takes the current state and returns the next state. The `Reason` node is called by the graph engine. The `Reason` node is responsible for generating the next message in the conversation. The `Reason` node is also responsible for handling the tool calls and for routing the graph to the appropriate tool node.

Agent Basics

Agents are programs like chatbots. They can make decisions from user input in natural language. They do not need human help. They can either call a tool or give a final answer.

Conditional Edges and Tool Nodes

These are the rules and points that guide the agent. They decide when the agent should use a tool and when it should stop and give the answer.

ReACT Architecture

This design has three clear stages:

- **Observe:** The agent looks at the tool's output. It does not end the chat yet.
- **Act:** The agent takes action. It calls a tool when needed.
- **Reason:** The agent thinks about the output. It decides the next step.

This loop can repeat until the agent is ready to give the final answer.

LangGraph ReACT Loop Notes

ReACT stands for Reason + Act + Observe + Answer. It is a structured way for an agent to handle queries by combining reasoning with tool use.

The loop ensures that the agent does not just guess answers but can call external tools, observe their outputs, and then respond with accurate information.

What is the ReACT Loop?

Step by Step in the BankBot Example

1. HumanMessage

The user input is wrapped as a `HumanMessage`. For example: "weather in pune". This tells the agent that the message came from the user.

2. Agent Node (Reason)

The LLM reads the `HumanMessage` and reasons about what to do. In this case, it realizes that weather data must come from a search tool. It generates a structured tool call request.

3. Tool Node (Act)

The graph routes to the `ToolNode` because the agent requested a tool. The tool executes the query, such as calling a search API for "weather in pune".

4. ToolMessage (Observe)

The tool's output is wrapped as a `ToolMessage`. This tells the agent that the information came from a tool. For example: "Current temperature in Pune is 34°C, partly cloudy."

5. Agent Node Again (Reason Again)

The LLM sees the `ToolMessage` and reasons again. Now it has the data, so it decides to summarize it clearly for the user. If more data was needed, it could loop back to call another tool.

6. AIMessage (Final Answer)

The agent produces the final structured reply as an `AIMessage`. For example: "The weather in Pune is 34°C with partly cloudy skies." This is what the user sees as the answer.

- `HumanMessage` = user input
- `Agent Node` = reasoning step
- `Tool Node` = action step
- `ToolMessage` = observation step
- `AIMessage` = final answer

This loop shows how LangGraph encodes the ReACT pattern:

Reason → Act → Observe → Reason Again → Answer.
It makes clear that the agent is not guessing but following a disciplined process.

